TECH BIZ SECURITY · SOC ANALYST INTERNSHIP · BATCH B-503

WEEK 4 · TASK 12

SIEM Dashboards & Alerts in Splunk

Submitted by	**Oscar ALIDJINOU**
Student ID	**TBS-INT-2026-2398**
Program	**SOC Analyst Internship (6 weeks)**
Company	**Tech Biz Security**
Date of submission	**11-11-2026**

01 Objective

Searching a SIEM by hand answers questions one at a time. A dashboard keeps the important answers on screen, and an alert watches the data for me and speaks up when something suspicious happens. This task made my Splunk lab useful in both ways: I built a dashboard with three panels from my failed-login searches, created an alert that triggers when there are more than five failed logins in ten minutes, and tested it until I saw it fire.

02 Proof of Completion

Room / activity	Duration	Tasks	Status
TryHackMe: Introduction to SIEM	120 min	7	Completed, room progress 100%
Lab: dashboard with three panels	-	-	Done (section 04)
Lab: alert for repeated failed logins	-	-	Created and triggered (sections 05 and 06)

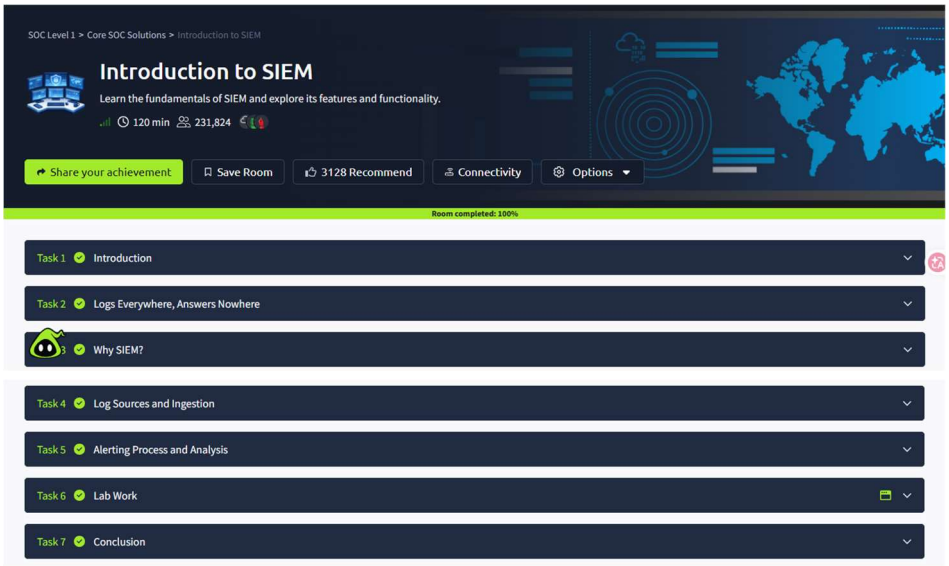


Figure 1. Introduction to SIEM: all 7 tasks completed (100%).

03 Key Concepts

3.1 Dashboards and visualisations

A dashboard is a page of panels, each one a saved search shown as a table or chart. The skill is choosing the right form for the question being asked.

Visualisation	Best for	Example
Statistics table	Exact values and rankings.	Failed logins per user.
Single value	One headline number.	Total failed logins.
Column or line chart	Change over time.	Failed logins per minute.
Pie or bar chart	Comparing shares of a total.	Which user is targeted most.

3.2 Alerts

An alert is a saved search that Splunk runs by itself and compares with a condition. If the condition is met, it triggers an action.

Element	Meaning
Alert type	Scheduled alerts run on a timetable (for example every minute). Real-time alerts watch continuously and cost more resources.
Trigger condition	The rule that decides if an alert fires, such as the number of results being greater than 5.
Throttle	A pause after a trigger, so the same incident does not raise a flood of identical alerts.
Trigger action	What happens when it fires: add to the Triggered Alerts list, send an email, run a script.
Severity	How urgent the alert is, so an analyst can prioritise.

> **Why more than 5 failures in 10 minutes?**
> One or two failed logins are usually a typing mistake, like the harmless 4625 event I analysed in Week 2. More than five within ten minutes looks like guessing. The threshold is a balance: too low floods the team with false positives, too high lets an attack slip through.

04 The Dashboard

I built one dashboard, SOC Lab Dashboard, with three panels. Each one started as a search, was turned into a visualisation with the Visualization tab, and was saved with Save As, Dashboard Panel. All searches used the Last 3 days time range.

#	Panel	Search	Visualisation
1	Failed logins over time	`index=main "Failed password" | timechart span=1m count`	Pie chart
2	Failed logins by user	`index=main "Failed password" | rex "for (?:invalid user )?(?<user>\S+) from (?<src_ip>\S+) port" | stats count by user | sort - count`	Statistics table
3	Total failed logins	`index=main "Failed password" | stats count`	Statistics table

Panel 1: failed logins over time

I ran the search, opened the Visualization tab and chose a chart, then saved it as a panel in a new dashboard named SOC Lab Dashboard, with Private permissions.

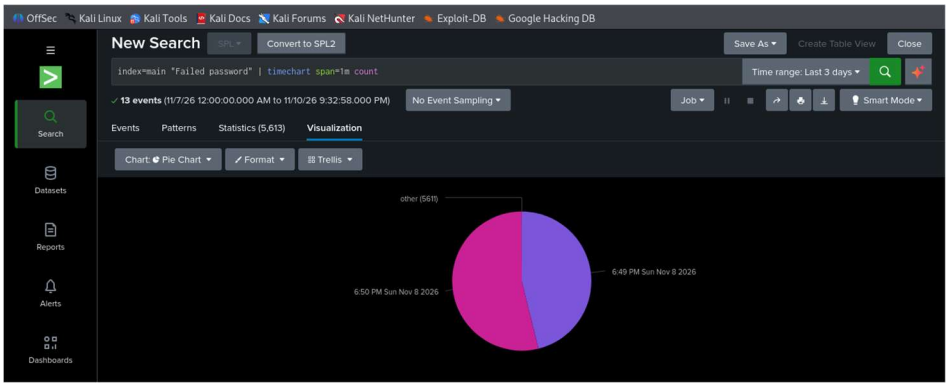


Figure 2. Panel 1: the timechart search shown as a visualisation.

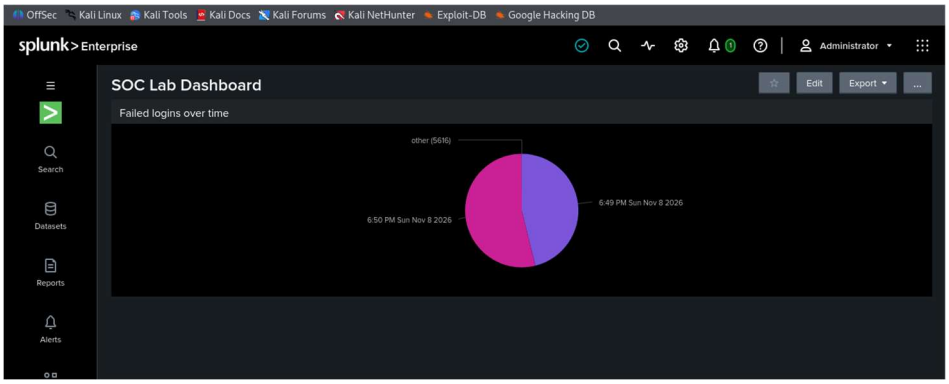


Figure 3. The new SOC Lab Dashboard with its first panel.

Panel 2: failed logins by user

The second search reuses the rex search from Task 11 to count failures per username. It shows the account test with 4 failures, and admin, oracle and root with 3 each, so 13 in total.

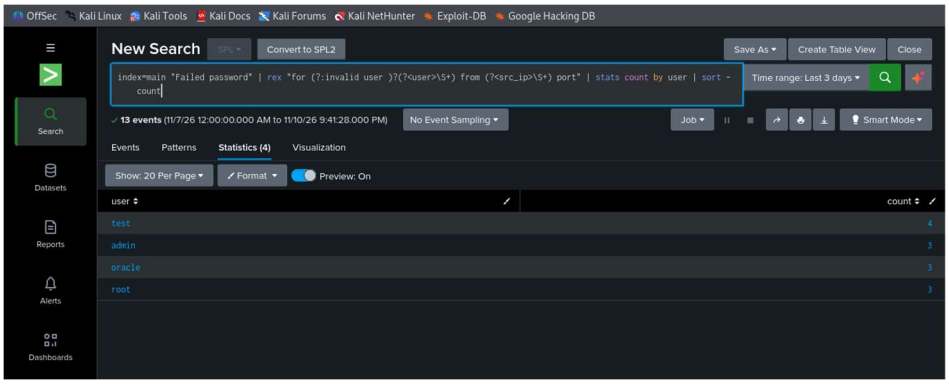


Figure 4. Panel 2: failed logins by user, as a statistics table.

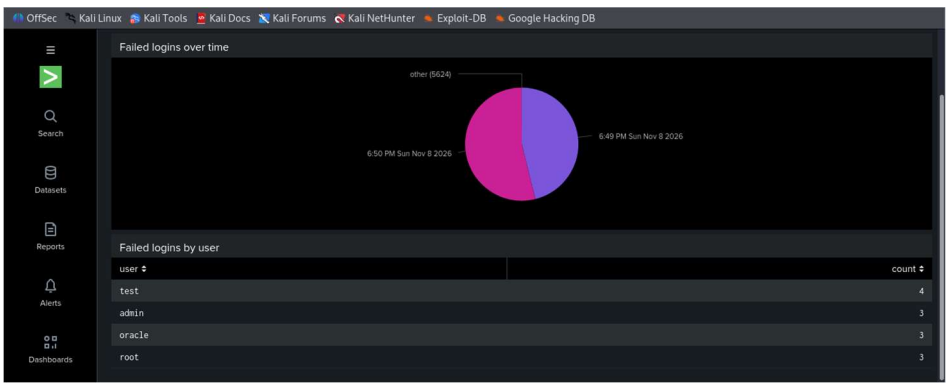


Figure 5. The dashboard with two panels.

Panel 3: total failed logins

A headline number answers the first question every analyst asks: how many? The count is 13.

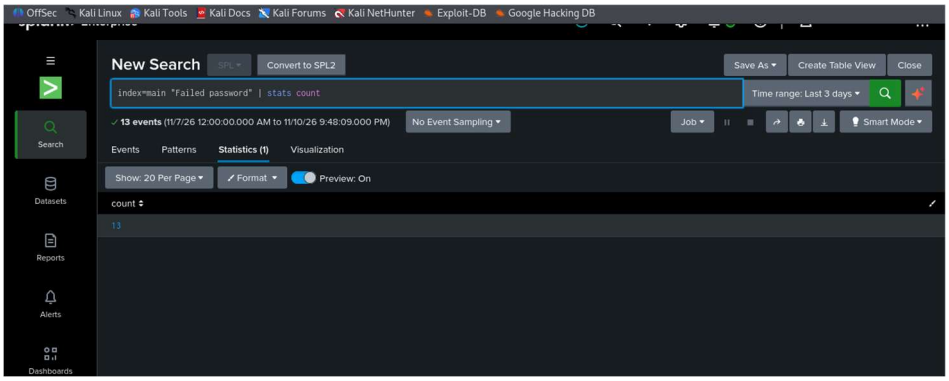


Figure 6. Panel 3: the total as a statistics table.

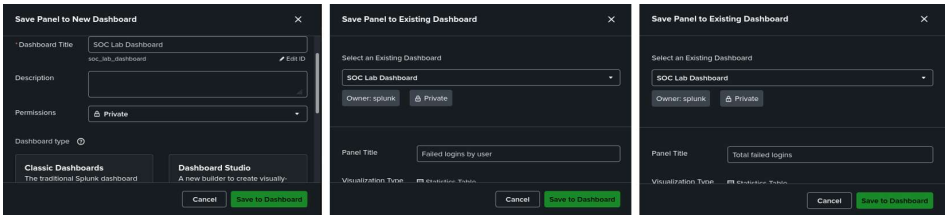


Figure 7. The Save Panel dialogs: new dashboard (panel 1) and existing dashboard (panels 2 and 3).

The finished dashboard



Figure 8. SOC Lab Dashboard with its panels: failed logins over time, by user, and the total.

> **What I would improve**
>
> A pie chart is the wrong tool for data over time, because it shows shares of a whole, not a trend. My panel 1 groups the data per minute across three days, and Splunk lumps almost every empty minute into one large "other" slice, so only two named slices (18:49 and 18:50) remain. A column chart over a narrower time range would show the burst of failures clearly. Choosing the visualisation that matches the question is the main lesson of this panel.

05 The Alert

I turned a search for failed passwords into an alert named Brute Force, using Save As, Alert. These are the settings I chose.

Setting	Value	Why
Title	Brute Force	A clear name for the analyst.
Description	More than 5 failed SSH logins in 10 min	States the rule in plain words.
Search	`index=main "Failed password"`	Finds every failed SSH password.
Time range	The last 10 minutes	Matches the "in 10 min" part of the rule.
Permissions	Private	Visible only to me in this lab.
Alert type	Scheduled, on a cron schedule	Runs the search on a timetable.
Cron expression	`* * * * *`	Every minute, so a test shows results quickly.
Expires	24 hours	How long the triggered alert record is kept.
Trigger alert when	Number of Results is greater than 5	The detection rule itself.
Trigger	Once	One alert per run, not one per event.
Throttle	Suppress triggering for 10 minutes	Avoids repeated alerts for the same burst.
Trigger action	Add to Triggered Alerts, severity Medium	Lists the alert in Splunk for the analyst.

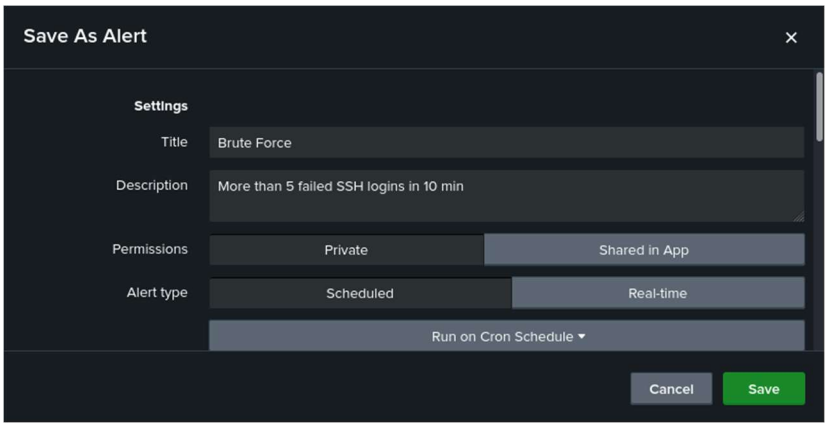


Figure 9. Save As Alert: title, description, permissions and alert type.

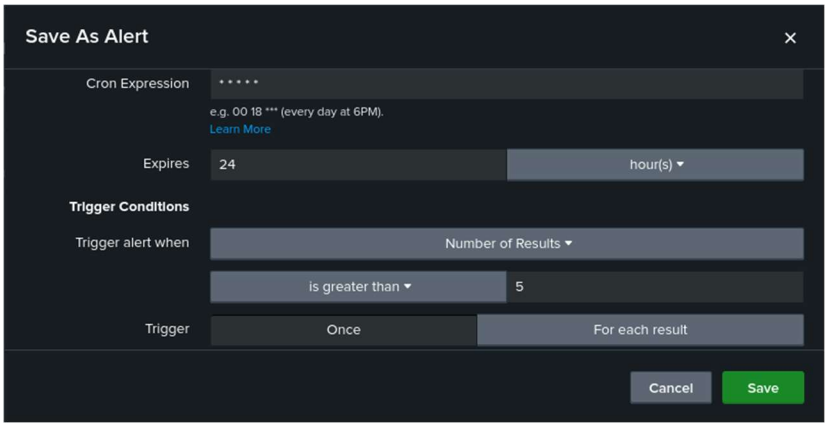


Figure 10. Cron schedule, expiry and the trigger condition (more than 5 results).

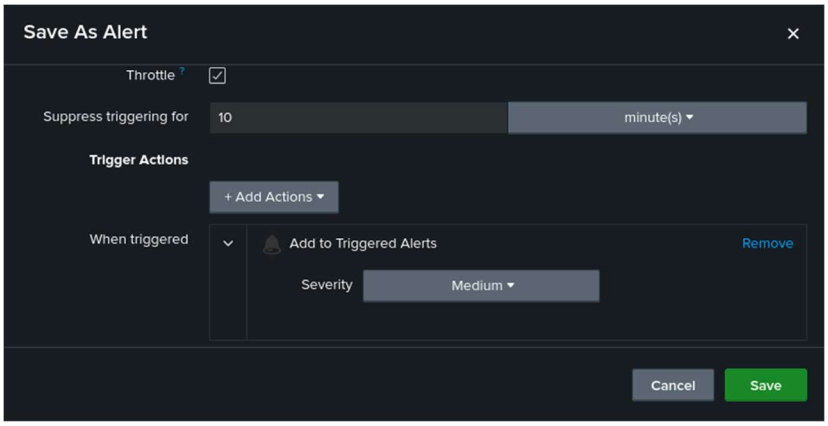


Figure 11. Throttle for 10 minutes and the action: Add to Triggered Alerts, severity Medium.

After saving, the alert's page confirms the configuration. At that moment, it reports no fired events, which is the correct state before any test.

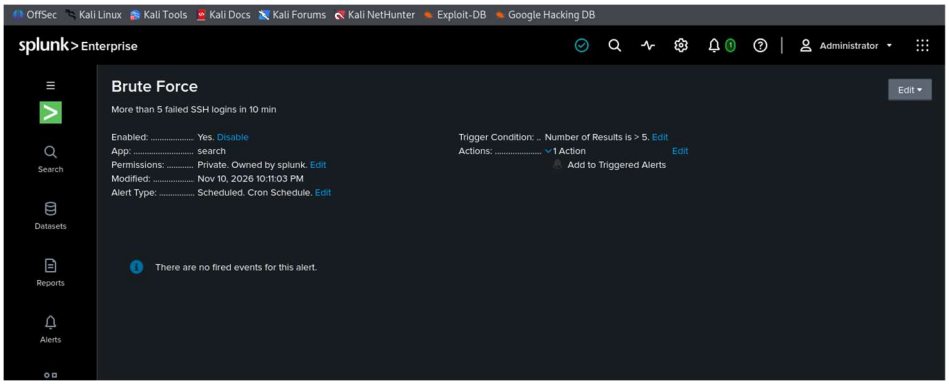


Figure 12. The saved Brute Force alert: enabled, scheduled, trigger condition Number of Results > 5.

06 Testing the Alert

6.1 Generating failed logins

I ran the same loop as in Task 11: four usernames (admin, root, test, oracle), three wrong-password attempts each, against my own machine. That makes 12 failures, more than the threshold of 5.

```
server_soc@ubuntu:~$ for u in admin root test oracle; do for i in 1 2 3; do sshpass -p 'wrong
pass' ssh -o StrictHostKeyChecking=no -o PubkeyAuthentication=no $u@127.0.0.1 true; done; don
e
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
server_soc@ubuntu:~$
```

Figure 13. Twelve failed SSH attempts, all ending in "Permission denied".

6.2 The alert fires

On its next run, the scheduled search found more than five results and triggered. The Triggered Alerts page lists it with its time, type, severity and mode.

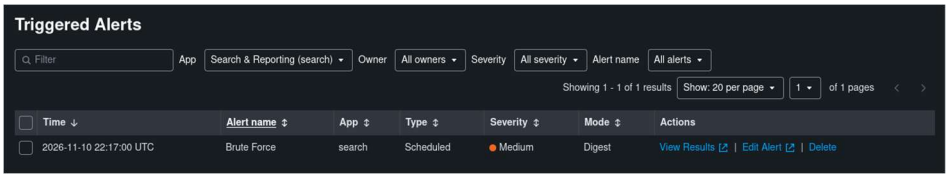


Figure 14. Triggered Alerts: Brute Force fired at 22:17:00 UTC, severity Medium.

6.3 The proof behind it

View Results opens the search that the alert ran, with the exact ten-minute window from 10:07 PM to 10:17 PM. It returned 12 events, which is exactly the 12 attempts I made.

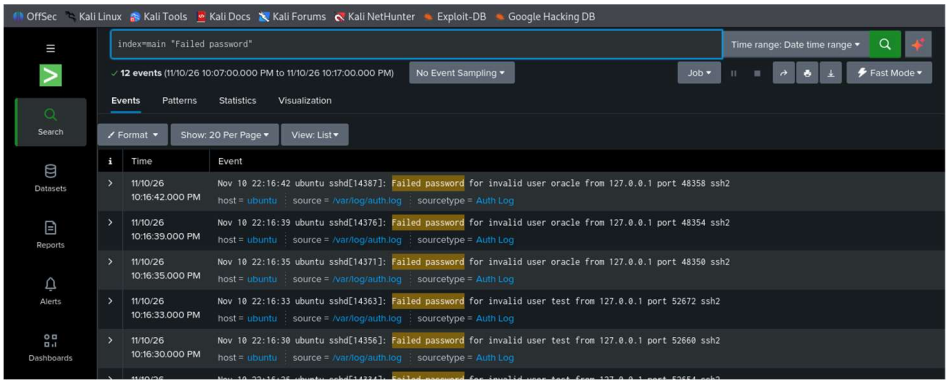


Figure 15. View Results: 12 failed-password events in the alert's 10-minute window.

Check	Result
Before the test	The alert's page showed no fired events.
The test	12 failed logins generated, above the threshold of 5.
After the test	A Triggered Alerts entry for Brute Force, severity Medium, at 22:17:00 UTC.
Events behind it	12 events in the window from 22:07 to 22:17, matching my 12 attempts.
Verdict	The alert works as designed.

Analyst assessment
The alert correctly detected twelve failed logins across several usernames in about a minute, a brute-force pattern. The source is 127.0.0.1, my own machine, so it is a test and not an incident. On a real server, the next steps would be to check the source address against threat intelligence, look for an "Accepted" line from it, and block it if needed. The pattern maps to MITRE ATT&CK T1110 Brute Force.

07 From Lab to a Real SOC

Improvement	Why it matters
Group by source address	Alert when one IP has more than five failures, instead of counting all failures together, so that unrelated users who mistype are not combined.
Watch for a success after failures	A successful login right after a run of failures is the dangerous case and deserves a higher severity.
Tune the threshold	Use the baseline of normal failures to set a number that avoids false positives.
Add a real action	Send an email or a chat message so that the analyst is notified without opening Splunk.
Document a response	Define what the analyst does when it fires: investigate, enrich, escalate or close.

08 Reflection and Key Takeaways

- A dashboard is only as good as its visualisation choices: the right chart makes a pattern obvious, and the wrong one hides it.
- An alert turns a search I ran by hand into a watch that never gets tired, but its threshold and throttle must be chosen with care.
- Testing an alert end to end, from generating events to seeing the trigger and its evidence, is the only way to trust it.
- The failed-login work of the past weeks, from grep to Wireshark to Splunk, now runs automatically.

Next step: replace the pie chart with a column chart, and add an alert that groups failures by source address.

09 References

- TryHackMe: Introduction to SIEM; Splunk documentation: dashboards, visualisations and alerting.